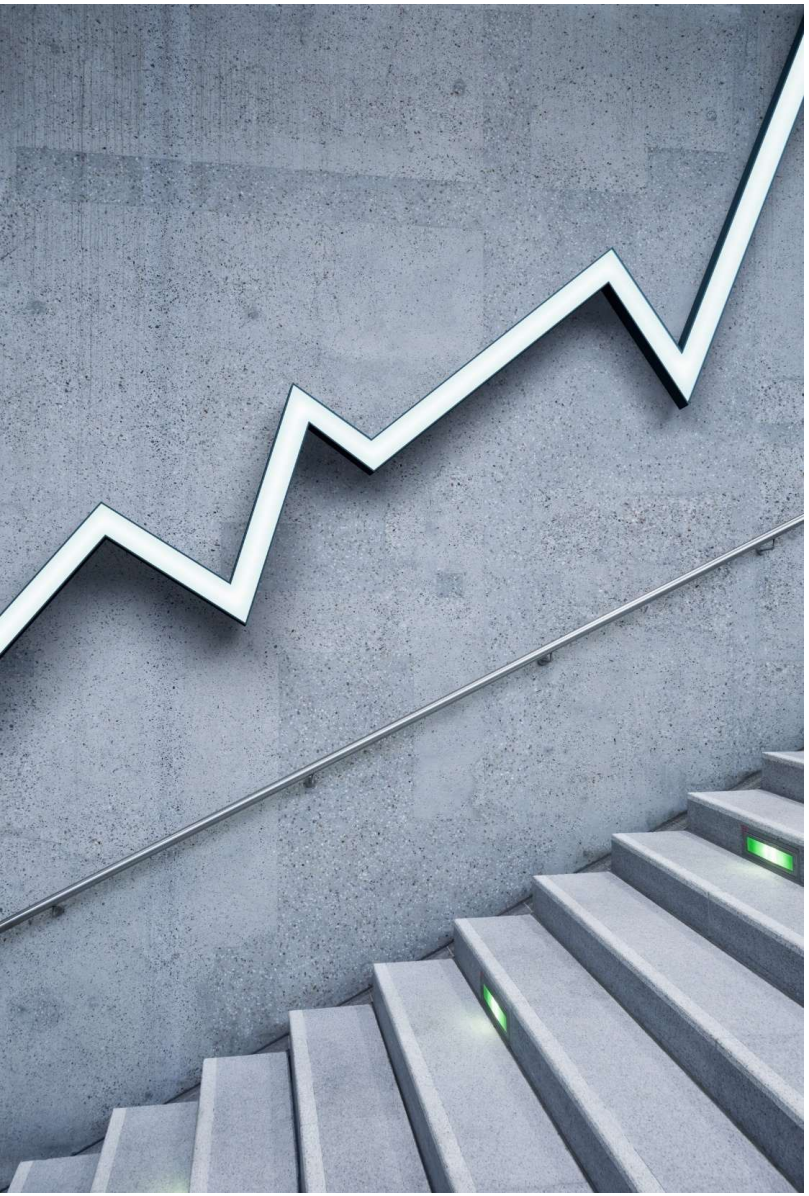


Learning Python

PRAVEEN NAIR

Part 1

PYTHON FUNDAMENTALS

Introduction to Python

1. Invented in the Netherlands, early 90s by Guido van Rossum named after Monty Python Flying Circus series
2. Open sourced from the beginning
3. Scalable, object oriented and functional from the beginning
4. Increasingly popular
5. Strong Libraries

What is Python used for?

1. Beginner-friendly programming language
2. Data Analysis and Machine Learning
3. Web & Desktop Application Development
4. Automation or Scripting
5. Programming Internet of Things (IoT) devices

Implementations of Python

CPython – the standard and most widely used implementation.

PyPy – a faster implementation with a Just-In-Time (JIT) compiler.

Jython – runs on the Java Virtual Machine (JVM).

IronPython – runs on the .NET framework.

MicroPython – designed for microcontrollers and embedded devices.

Installing Python…

1. Visit https://www.python.org/downloads/
2. Click on Download Python button
3. Run the installer (make sure to tick add path checkbox)

Installing Visual Studio Code…

1. Visit https://code.visualstudio.com/download
2. Click on Download button
3. Run the installer



Variables

We use variables to temporarily store data in computer's memory.

price = 10 (Integer) #type(price)

rating = 4.9 (Python represent float values as 64-bit "double-precision" values so no separate type as double)

course_name = 'Python for Beginners' (String)

is_published = True (Boolean)

Data Structures – List, Tuple, Dictionary, Set, Array (A data structure is a specialized format for organizing, processing, retrieving and storing data.)

……

Type function, multiple assignment, variable naming rule

Multi-line String

To define a multi-line string, we surround our string with tripe quotes.

```
msg = '''
Dear Friend,
How are you? Hope all is well with you.
Thanks,
MyName
'''
print(msg)
```

Type Conversion

```
birth_year = input("Enter your birth year")
age = 2022 - int(birth_year)
print(age)
---------------
 float()
 Eval()
```

Printing Variables

```
name="John"

print(name)
```

Receiving Input

```
name=input("Enter your name")

print(name)
```

Concatenate Strings

```
first_name = "John "

last_name = "Smith "

print(first_name + last_name)
```

Formatted String Literals (f-string)

```
name=input("Enter Name ")
favorite_color=input("Enter Favorite Colour ")
msg = f'{name} likes {favorite_color}’
print(msg)
```

Comments

```
birth_year = input("Enter your birth year")

age = 2022 - int(birth_year)

# print(age)

Print(birth_year)

---------

How to comment multiple lines?
```

Arithmetic Operations

```
+
-
*
/  returns a float
//  returns an int
%  returns the remainder of division
** exponentiation
```

Comparison Operators in Python

```
Equals: a == b

Not Equals: a != b

Less than: a < b

Less than or equal to: a <= b

Greater than: a > b

Greater than or equal to: a >= b
```

Logical Operators in Python

```
and: (a < b) and (a < c)

or: (a < b) or (a < c)

Not: not(a > c)
```



Python Escape Characters

```
txt="This is John\'s book." # \' means '

print(txt)

txt="This is a \n dummy text." # new line

print(txt)

txt="Name \t City \t Age" # tabspace

print(txt)
```

if...statement

```
a=10
b=15
if(a>b):
    print("a is big")
else:
    print("b is big")
```

if…elif…statement

```
a=10
b=10
if(a>b):
    print("a is greater")
elif(b>a):
    print("b is greater")
else:
    print("Both a and b are equal")
```

Switch/Match statement…

```
subject="HTML"
match subject:
    case "HTML":
        print("HyperText Markup Language")

    case "CSS":
        print("Cascading Style Sheet")
    case _:
        print("Unknown")
```

Printing with end parameter

By default, the print function ends with a newline.

Passing the whitespace to the end parameter (end=‘ ‘) indicates that the end character has to be identified by whitespace and not a newline.

```
----------------------

print("Hello", end=' ')

print("World")
```

While loops

```
i = 1
while i < 5:
    print(i)
    i += 1
------------------
Print Series –
5,10,15,20,25,30.....
```

While loops – break statement

```
secret_number=5
guess_count=0
guess_limit=3

while(guess_count < guess_limit):
    guess = int(input("Guess the number: "))
    guess_count = guess_count + 1
    if (guess==secret_number):
        print("You Win!")
        break
else:
    print("Better luck next time!")
```

While loops – continue statement

```
i=0
while (i<5):
  i = i + 1
  if (i==3):
    continue
  print(i)
```

While loops – pass statement

```
i=0
while (i<5):
  i = i + 1
  if (i==3):
    pass
  else:
    print(i)
```

For loops

```
for i in range(5): # iterates over a sequence
    print("Hello")
--------------
```

- `range(5)`: generates 0, 1, 2, 3, 4
- `range(1, 5)`: generates 1, 2, 3, 4
- `range(1, 5, 2)`: generates 1, 3

For loop with else

```
n = 23
for i in range(2,n):
    if n % i == 0:
        print("Not a prime number")
        break
else:
    print("Prime Number")
.....................
```

Try with while..else also

Nested For loop

```
for i in range(5):
  for j in range(5):
      print(j)
```

Assignment: Student Information and Marks

Write a Python program that accepts the following details from the user using **Scanner**:

1. Student name
2. Student age
3. Student roll number
4. Marks in Java
5. Marks in Python
6. Marks in Database

Calculate and display:

1. Student details
2. Total marks
3. Average marks

No if or loops or functions

Assignment: Employee Salary Calculator

Write a Python program that accepts the following details from the user:

1. Employee name
2. Employee ID
3. Basic salary
4. HRA
5. DA

Calculate and display:

1. Employee details
2. Gross salary = Basic Salary + HRA + DA
3. Annual salary = Gross Salary × 12

No if or loops or functions

Assignment: Shopping Bill Calculator

Write a Python program that accepts:

1. Customer name
2. Product name
3. Product price
4. Quantity

Calculate and display:

1. Total price
2. Discount of 10%
3. Final bill amount after discount

No if or loops or functions

Student Result & Grade Calculator

Create a Python program that accepts a student's marks in **5 subjects**, calculates the total and percentage, and determines:

Pass or Fail

Grade

Division

Scholarship eligibility

Note: Do not use loops, arrays, methods, classes beyond the main class, or switch

Use if statement

Requirements

The student enters:

Student name

Java marks

Python marks

Database marks

Web Technology marks

Aptitude marks

Each subject is out of 100.

Condition 1 - Validate marks

If any mark is:

Less than 0 ⟶ Invalid marks

Greater than 100 ⟶ Invalid marks



Condition 2 - Pass/Fail

A student passes only if **each subject is at least 40.**

Otherwise:

Result: FAIL

Reason: One or more subjects below 40

Condition 3 - Grade

Percentage	Grade
90–100	A+
80–89	A
70–79	B
60–69	C
50–59	D
40–49	E

Condition 4 - Division

Percentage	Division
75+	Distinction
60–74	First Class
50–59	Second Class
40–49	Third Class

Condition 5 - Scholarship

Display:
Scholarship: Eligible
only when:
percentage >= 85 AND every subject >= 75
Otherwise:
Scholarship: Not Eligible

Sample Output

```
===== STUDENT RESULT SYSTEM =====
Enter student name: Rahul
Enter Java marks: 85
Enter Python marks: 78
Enter Database marks: 92
Enter Web Technology marks: 88
Enter Aptitude marks: 81
===== RESULT =====
Student Name : Rahul
Total        : 424 / 500
Percentage   : 84.8%
Result       : PASS
Grade        : A
Division     : First Class
Scholarship  : Not Eligible
```

Bank Management System

Create a **console-based Bank Management System** in Python. The program should allow a user to deposit money, withdraw money, check the current balance, and exit the application.

Initial Balance

The account should start with an initial balance of **₹5,000**.

Main Menu

When the program starts, display the following menu:

===== BANK MANAGEMENT SYSTEM =====

1. Deposit
2. Withdraw
3. View Balance
4. Exit

The menu should continue to appear until the user selects **Exit**.

Use if and loop if necessary

Expected Output

```
===== BANK MANAGEMENT SYSTEM =====
1. Deposit
2. Withdraw
3. View Balance
4. Exit

Enter your choice: 1
Enter deposit amount: 2000
Amount deposited successfully.
New Balance: 7000.0

===== BANK MANAGEMENT SYSTEM =====
1. Deposit
2. Withdraw
3. View Balance
4. Exit

Enter your choice: 4
Thank you for using the Bank Management System.
```

String is a collection of Characters

```
name="John Smith"

print(name[0])



--------------------

Print(name[-1]
```

Basic String Methods

```
name = "John“
len(name)
print(name.upper())
print(name.lower())
name.swapcase()
print(name.title()) – first letter of each word
name.capitalize() – first letter
print(name.find('n'))
print(name.replace('John', 'Ron'))
```

Searching - String Methods

```
name = "John“

print(name.find('n'))

print(name.index(‘h’)) #error if not found

Print(name.count(‘o’)

name.startswith('H’)

name.endswith('n')
```

validation - String Methods

```
name = "John“

print(name.isalnum()) # alphanumeric

print(name.isalpha()) # alphabets

print(name.isdigit()) # digits

print(name.islower())

print(name.isupper())

print(name.isspace())

print(name.istitle())
```

modifying - String Methods

```
name=" Hello World "

print(name.replace("Hello","Hi"))

print(name.strip())

print(name.lstrip())

print(name.rstrip())
```



Joining/splitting - String Methods

```
name="Hello World"

print(name.split(" "))

print(name.rsplit("o",1))

print(name.splitlines())

print("-".join(name))

print("-".join(new)) #new = name.split()
```

Task 1

Write a Python program to:

Take a string input from the user.

Print it in **uppercase**, **lowercase**, and **title case**.

Count how many **characters** are in it (excluding spaces).

Hint: Use .upper(), .lower(), .title(), and .replace(" ", "").

Solution 1

```
text = input("Enter a string: ")

print("Uppercase:", text.upper())

print("Lowercase:", text.lower())

print("Title case:", text.title())

count = len(text.replace(" ", ""))

print("Character count (excluding spaces):", count)
```

Task 2

Ask the user to enter a username.
Check if:

It contains only alphabets or numbers.

It is **between 5 and 15 characters** long.

Solution 2

```
username = input("Enter a username: ")
if username.isalnum() and len(username) <= 15 and len(username) > 5:
    print("Valid Username")
else:
    print("Invalid Username")
```

Task 3

Input a sentence from the user and:

Count how many words are there.

Print the **first** and **last** word.

Convert the sentence into **title case**.

Hint: Use .split() and .title().

Solution 3

```
sentence = input("Enter a sentence: ")

words = sentence.split()

print("Number of words:", len(words))

print("First word:", words[0])

print("Last word:", words[-1])

print("Title Case:", sentence.title())
```

Task 4

Take a paragraph as input.

Ask the user for a word to **find** and a word to **replace it with**.

Print the new paragraph.

Show how many times the word was replaced.

Hint: Use .count() and .replace().

Solution 4

```
paragraph = input("Enter a paragraph: ")

find_word = input("Enter word to find: ")

replace_word = input("Enter word to replace with: ")

count = paragraph.count(find_word)

new_para = paragraph.replace(find_word, replace_word)

print(new_para)

print(f"\n'{find_word}' was replaced {count} time(s).")
```

List – stores list of items in square bracket

```
numbers = [1, 2, 3, 4, 5]
numbers[0] # returns the first item
numbers[1] # returns the second item
numbers[-1] # returns the first item from the end
numbers[-2] # returns the second item from the end
--------------
Numbers[0] = 9
print(numbers)
numbers[1:3]
```

List – using ‘for loop’

```
items = ['apple','orange','mango']
for i in items:
    print(i)
```

List – Methods

```
numbers = [3,4,5,4,3]
numbers.append(6) # adds 6 to the end
numbers.extend([7,8])
numbers.insert(0, 6) # adds 6 at index position of 0
numbers.remove(6) # removes 6
numbers.pop() # removes the last item
numbers.pop(4) #removes 5th element
print(numbers.count(4))
numbers.clear() # removes all the items
numbers.index(8) # returns the index of 1st occurrence of 8
numbers.sort() # sorts the list –reverse=True
numbers.reverse() # reverses the list
```

List – Builtin functions

```
numbers = [5,8,4,3]

print(len(numbers))

print(max(numbers))

print(min(numbers))

print(sum(numbers))

print(sorted(numbers)) #can assign in new variable unlike sort
```

Task 1

Take a list of 5 numbers as input.

Sort them in **ascending** order.

Then reverse the order.

Print both results.

Hint: Use sort() and reverse().

Solution 1

```
numbers = [5,2,1,2,9]

numbers.sort()

print(numbers)

numbers.reverse()

print(numbers)
```

Task 2

Take two lists of numbers.

Merge them into one list.

Sort the combined list.

Hint: Use extend() and sort().

Solution 2

```
num1 = [4,1,5]

num2 = [7,8,6]

num1.extend(num2)

print(num1)

num1.sort()

print(num1)
```

Task 3

Create a list with duplicate elements and remove all duplicates while keeping order.

Example:
Input ⟶ [1, 2, 2, 3, 1, 4]
Output ⟶ [1, 2, 3, 4]

Hint: Use a for loop and check if element already exists in a new list.

Solution 3

```
num1 = [1, 2, 2, 3, 1, 4]
num2 = []
for i in num1:
    if i not in num2:
        num2.append(i)
print(num2)
```

Task 4

Take a list of numbers.

Create two new lists:

- One for even numbers.
- One for odd numbers.

Print both lists.

Hint: Use a for loop and % operator.

Tuple

They are like read-only lists in round bracket. We use them to store a list of items. But once we define a tuple, we cannot add or remove items or change the existing items.

```
----------------------

items = ('apple','orange','mango')

print(items[0])

items[0] = 'cherry’ - Error

----------------------

Try above with a list
```

Tuple Methods

```
customer = (3,4,5,4)

print(customer.index(4))

print(customer.count(4))
```

Tuple – Builtin function

```
t= (1,2,3)
len(t)      Number of elements
max(t)      Maximum value
min(t)      Minimum value
sum(t)      Sum of elements
sorted(t)   Returns a sorted list (not tuple)
tuple(iterable) Converts iterable to tuple
```

Set – Definition

Set is a collection which is unordered, unchangeable, and unindexed. No duplicate members. But we can remove and add items. Curly brackets.

```
items = {'apple','orange','mango'}
```

Set - example

```
items = {'apple','orange','mango'}

for i in items:

    print(i)

----------------------

Try print(items[0])
```

Set - methods

```
Adds element x to the set  s.add(10)
Adds multiple elements     s.update([4,5,6])
Removes element x (error if not found)s.remove(3)
Removes element if exists (no error)  s.discard(3)
Removes and returns a random element  s.pop()
Removes all elements  s.clear()
```

Frozen Set

```
Check diff between set and frozen set

…….

a=[1,2,3]
s=frozenset(a)

s.add(9) – will throw error
print(s)
```

Dictionaries

We use dictionaries to store key/value pairs.

```
customer = {
"name": "John",
"age": 20,
"city": "california"
}
print(customer["name"])
```

Dictionaries – key, value

```
customer = {
    "name": "Alice",
    "age": 25,
    "city": "London"
}
for key, value in customer.items():
    print(key + " - " + value)
```

Dictionaries methods

```
customer.clear()
print(customer["name"]) #error if doesn’t exist
print(customer.get("name")) #none if doesn’t exist
print(customer.get("name“,0)) #0 if doesn’t exist
print(customer.items())
print(customer.keys())
print(customer.values())
customer.pop('age')
customer.popitem() #last item
del customer[‘age’]
print(customer)
customer.update({"phone":343435455})
print(customer)
```

Dictionaries – builtin functions - values

```
result = {
    "maths": 3,
    "english": 25,
    "science": 5
}
print(max(result,key=result.get))
print(min(result,key=result.get))
print(sum(result.values()))
```

Dictionaries – builtin functions - keys

```
result = {
    23: 3,
    12: 25,
    11: 5
}
print(max(result))
print(min(result))
print(sum(result))
```

Task 1

Create a dictionary called student with the following data:

name = "John“
age = 20
course = "Python“
marks = 85

Perform the following:

Print all keys and values.
Add a new key "email": "john@example.com".
Update the marks to 90.
Remove the "course" key.
Print the final dictionary.

Solution 1

```
student = {
    "name": "John",
    "age": 20,
    "course": "Python",
    "marks": 85
}
print("Initial:", student)

student["email"] = "john@example.com"   # Add

student["marks"] = 90                   # Update

student.pop("course")                   # Remove

print("Final:", student)
```

Task 2

Write a program that counts how many times each word appears in a given sentence.

Example:

sentence = "python is fun and python is easy"

Output:

{'python': 2, 'is': 2, 'fun': 1, 'and': 1, 'easy': 1}

Hint: Use .split() and a dictionary to count.

Solution 2

```
sentence = "python is fun and python is easy"

words = sentence.split()

word_count = {}

for word in words:

    word_count[word] = word_count.get(word, 0) + 1

print(word_count)
```

Task 3

Create a dictionary with student names as keys and their marks as values:

marks = {"Alice": 85, "Bob": 90, "Charlie": 78}

Do the following:

Print all student names and marks.

Find and print the student with the highest marks.

Add a new student "David": 88.

Calculate and print the **average** marks.

Solution 3

```
marks = {"Alice": 85, "Bob": 90, "Charlie": 78}
for name, score in marks.items():
    print(name,  score)
topper = max(marks, key=marks.get)
print("Topper:", topper)
marks["David"] = 88
average = sum(marks.values()) / len(marks)
print("Average Marks:", average)
```

Task 4

Given two dictionaries:

dict1 = {"a": 1, "b": 2}

dict2 = {"b": 3, "c": 4}

Create a new dictionary that merges both, where common keys have their values **added together.**
Output:

{'a': 1, 'b': 5, 'c': 4}

Solution 4

```
dict1 = {"a": 1, "b": 2}

dict2 = {"b": 3, "c": 4}

merged = dict1.copy()

for key, value in dict2.items():

    merged[key] = merged.get(key, 0) + value

print(merged)
```

Task 5

Create a dictionary to store product stock:

inventory = {"apple": 10, "banana": 5, "orange": 8}

Write a program to:

Add a new product.

Update stock quantity.

Check if a product is available.

Print all available products.

Solution 5

```
inventory = {"apple": 10, "banana": 5, "orange": 8}
inventory["grape"] = 12
inventory["banana"] += 3
item = "apple“
if item in inventory:
    print(f"{item} available ({inventory[item]} left)")
else:
    print(f"{item} not found")

for product, qty in inventory.items():
    print(product, "→", qty)
```

List vs Tuple vs Set vs Dictionary

List is a collection which is ordered and changeable. Allows duplicate members. Square brackets.

Tuple is a collection which is ordered and unchangeable. Allows duplicate members. Round brackets.

Set is a collection which is unordered, unchangeable, and unindexed. No duplicate members. But we can remove and add items. Curly brackets.

Dictionary is a collection which is ordered and changeable. No duplicate members. Curly brackets.